

Projet CSI Sécurité Sociale — Présentation complète

1. Contexte général

Ce projet est une application de gestion d'assurance santé appelée **Care Health** ou **CSI Sécurité Sociale**. Il s'agit d'une solution de type microservices avec un backend Java Spring Boot divisé en plusieurs services métier et une interface frontend en **React / TypeScript / Vite**.

L'objectif principal est de gérer : - l'authentification des utilisateurs, - la gestion des profils assurés, médecins et agents sociaux, - l'enregistrement des consultations et des prescriptions, - la génération et le suivi des feuilles de maladie, - le calcul, l'approbation et l'exécution des remboursements.

Ce document décrit le projet de bout en bout pour un formateur qui ne dispose pas du code.

2. Architecture globale

L'architecture repose sur un **API Gateway** et plusieurs microservices communiquant par HTTP. Un service de découverte (Eureka) est utilisé pour l'orchestration des services.

```
frontend React / Vite :5173
    |
    v
api-gateway :8080
    |
    +-- auth-service :8081 ----- auth-db
    +-- profile-service :8082 ----- profile-db
    +-- medical-service :8083 ----- medical-db
    +-- reimbursement-service :8084 --- reimbursement-db
    +-- discovery-service :8761
```

Les composants du projet sont : - api-gateway - auth-service - profile-service - medical-service - reimbursement-service - discovery-service - common - frontend

3. Services backend

3.1 common

Module partagé contenant les éléments transverses : - exceptions métiers, - DTO partagés, - sécurité JWT commune, - structures de réponse API standard.

3.2 auth-service

Fonctionnalités : - gestion des comptes utilisateurs, - login / refresh token, - génération de JWT, - inscription des nouveaux acteurs.

Rôle principal : centraliser l'authentification et émettre les tokens JWT qui protègent toutes les APIs.

3.3 profile-service

Fonctionnalités : - gestion des assurés, - gestion des médecins (généralistes, spécialistes), - gestion des agents sociaux, - affectation du médecin traitant, - consultations de profils, - dashboards statistiques pour les agents.

Ce service contient aussi les paramètres applicatifs et les tableaux de bord liés aux profils.

3.4 medical-service

Fonctionnalités : - enregistrement des consultations, - création et gestion des prescriptions, - génération des orientations vers spécialistes, - création des feuilles de maladie, - workflow des feuilles de maladie, - génération de PDF pour les feuilles.

Ce service est centré sur le parcours médical : médecin, consultation, feuille de soin.

3.5 reimbursement-service

Fonctionnalités : - création des demandes de remboursement, - calcul automatique des montants remboursés, - approbation ou rejet des demandes, - exécution du paiement, - génération des reçus.

Le calcul métier prévoit : - remboursement à 100 % pour les consultations généralistes, - remboursement à 80 % pour les consultations spécialistes.

3.6 api-gateway

Fonctionnalités : - point d'entrée unique pour toutes les APIs, - routage des requêtes vers les services appropriés, - sécurisation des routes, - intégration avec le service de découverte.

3.7 discovery-service

Service Eureka utilisé comme annuaire pour permettre aux autres services de se découvrir et de se connecter dynamiquement.

4. Base de données et stockage

Chaque service backend a sa propre base Postgres : - `auth-db` pour `auth-service`, - `profile-db` pour `profile-service`, - `medical-db` pour `medical-service`, - `reimbursement-db` pour `reimbursement-service`.

Les bases sont définies dans `docker-compose.yml` et séparées pour respecter l'isolation des domaines métier.

5. Sécurité et authentification

5.1 JWT

L'application utilise des **JWT** pour l'authentification. Les utilisateurs se connectent via : - `POST /api/v1/auth/login` - `POST /api/v1/auth/refresh`

Les tokens sont envoyés dans l'en-tête `Authorization: Bearer <token>`.

5.2 Gestion des rôles

Les rôles supportés sont : - `AGENT`, - `SOCIAL_AGENT`, - `AGENT_SOCIAL`, - `SECURITY_AGENT`, - `DOCTOR`, - `GENERALIST`, - `SPECIALIST`, - `ADMIN`.

Les rôles médicaux `GENERALIST` et `SPECIALIST` impliquent le rôle applicatif `DOCTOR`.

5.3 Inscription métier

Lors d'une inscription (`POST /api/v1/auth/register`), l'utilisateur est créé dans `auth-service`, puis un appel interne sécurisé vers `profile-service` crée le profil métier correspondant.

Exemples de profils métier : - agent social, - médecin généraliste, - médecin spécialiste.

6. Fonctionnalités métier principales

6.1 Gestion des assurés

On peut : - créer un assuré, - rechercher un assuré, - modifier un assuré, - changer son statut, - affecter un médecin traitant, - consulter l'historique du médecin traitant.

Le numéro d'assurance est unique et un assuré inactif ne peut pas avoir de nouvelle consultation.

6.2 Gestion des médecins

On gère : - les médecins généralistes, - les spécialistes, - leur statut actif/inactif, - les spécialités obligatoires pour les spécialistes.

Un médecin ne peut agir que sur ses propres dossiers et la relation `authUserId` -> `doctor` est utilisée pour l'autorisation.

6.3 Consultations

Une consultation est liée à un patient assuré et à un médecin. Les règles métier principales sont : - coût strictement positif, - date de fin après la date de début, - le médecin doit être l'auteur de la consultation.

6.4 Prescriptions et orientations

Une prescription est attachée à une consultation existante. Chaque ligne de prescription contient : - nom du médicament, - posologie, - fréquence, - durée, - quantité, - instructions optionnelles.

Un généraliste peut créer des orientations vers un spécialiste actif de la spécialité demandée.

6.5 Feuilles de maladie

Le workflow des feuilles de maladie est :

```
DRAFT -> ISSUED -> SUBMITTED -> UNDER_REVIEW -> APPROVED -> PAID
                                     |               |
                                     +--> REJECTED  +--> CANCELLED
```

- le médecin crée et émet la feuille,
- l'agent social la soumet au contrôle,
- l'agent la complète avec paiement et réception,
- une feuille payée ne peut plus être modifiée.

6.6 Remboursements

Cycle autorisé : - PENDING -> APPROVED -> EXECUTED - PENDING -> REJECTED

Règles importantes : - le rejet nécessite un motif, - seule une demande approuvée peut être exécutée, - l'exécution est idempotente, - une feuille ne peut avoir qu'une demande de remboursement.

6.7 Audit

Les actions sensibles sont journalisées dans un événement append-only contenant : - `actorUserId`, - `actorRole`, - `action`, - `resourceType`, - `resourceId`, - `timestamp`, - `result`, - métadonnées non sensibles.

7. API principales

Auth

- POST `/api/v1/auth/login`

- POST /api/v1/auth/refresh
- POST /api/v1/auth/register

Profil

- POST /api/v1/insured
- GET /api/v1/insured/{insuranceNumber}
- GET /api/v1/insured/{insuranceNumber}/status
- PUT /api/v1/insured/{insuranceNumber}/treating-doctor
- PUT /api/v1/insured/{insuranceNumber}
- PATCH /api/v1/insured/{insuranceNumber}/status
- POST /api/v1/insured/{insuranceNumber}/primary-doctor
- GET /api/v1/insured/{insuranceNumber}/primary-doctor-history
- POST /api/v1/doctors
- GET /api/v1/doctors/{matricule}
- GET /api/v1/doctors
- GET /api/v1/agents

Dashboards

- GET /api/v1/dashboard/agent-social/summary
- GET /api/v1/dashboard/agent-social/patients-stats
- GET /api/v1/dashboard/agent-social/doctors-stats
- GET /api/v1/dashboard/agent-social/reimbursements-stats
- GET /api/v1/dashboard/agent-social/recent-activities
- GET /api/v1/dashboard/doctor/summary
- GET /api/v1/dashboard/doctor/patients-stats
- GET /api/v1/dashboard/doctor/consultations-stats
- GET /api/v1/dashboard/doctor/disease-sheets-stats
- GET /api/v1/dashboard/doctor/recommendations-stats
- GET /api/v1/dashboard/doctor/recent-activities

Paramètres

- GET /api/v1/settings
- GET /api/v1/settings/{category}
- PUT /api/v1/settings/{category}

Medical

- POST /api/v1/consultations
- POST /api/v1/prescriptions/medications
- POST /api/v1/prescriptions/specialist-consultations
- POST /api/v1/consultations/{consultationId}/referrals
- GET /api/v1/referrals/{referralNumber}
- PATCH /api/v1/referrals/{referralNumber}/status
- POST /api/v1/disease-sheets

- GET /api/v1/disease-sheets/{sheetNumber}
- PATCH /api/v1/disease-sheets/{sheetNumber}/submit
- PATCH /api/v1/disease-sheets/{sheetNumber}/complete
- GET /api/v1/disease-sheets/{sheetNumber}/pdf

Remboursement

- POST /api/v1/reimbursements
- GET /api/v1/reimbursements/{reimbursementNumber}
- POST /api/v1/reimbursements/{reimbursementNumber}/calculate
- PATCH /api/v1/reimbursements/{reimbursementNumber}/approve
- PATCH /api/v1/reimbursements/{reimbursementNumber}/reject
- POST /api/v1/reimbursements/{reimbursementNumber}/execute

8. Frontend

Le frontend est une application React moderne en **TypeScript** et **Vite**. Il utilise : - React Router, - Axios, - TanStack Query, - React Hook Form, - Zod, - Tailwind CSS v4, - Zustand, - Lucide React, - React Hot Toast.

Il contient des éléments suivants : - **src/api** : client HTTP Axios et services REST, - **src/components** : composants réutilisables, - **src/hooks** : hooks personnalisés, - **src/layouts** : layouts d'authentification et d'application, - **src/pages** : pages métier, - **src/routes** : configuration des routes et des menus, - **src/store** : état global d'authentification, - **src/types** : interfaces TypeScript, - **src/utils** : helpers.

8.1 Fonctionnalités du frontend

- écran de connexion,
- création de compte,
- dashboard médecin,
- dashboard agent,
- gestion des patients couverts,
- gestion des médecins,
- création de consultations,
- gestion des prescriptions,
- suivi des feuilles de maladie,
- gestion des remboursements.

8.2 Thèmes et interface

L'application propose plusieurs thèmes avec persistance locale : clair, sombre, bleu médical, orange, violet, etc.

8.3 Stockage des tokens

Les tokens JWT sont stockés en `sessionStorage` via Zustand. Le frontend inclut un mécanisme de refresh automatique sur 401.

9. Exemples de scénarios métiers

9.1 Création de compte et synchronisation métier

1. L'utilisateur s'inscrit via `POST /api/v1/auth/register`.
2. `auth-service` crée le compte dans sa base.
3. `auth-service` appelle `profile-service` avec un secret interne pour créer le profil métier.
4. Le profil métier est disponible immédiatement dans les listes.

9.2 Affectation du médecin traitant

1. Un agent sélectionne un assuré et un généraliste actif.
2. L'ancien médecin traitant est clos.
3. Une nouvelle affectation est créée avec date de début.
4. L'historique reste consultable.

9.3 Orientation vers un spécialiste

1. Le généraliste crée une orientation depuis une consultation.
2. Il choisit une spécialité et un ou plusieurs spécialistes actifs.
3. L'orientation démarre en statut `PENDING`.
4. L'agent ou le spécialiste peut ensuite actualiser le statut.

9.4 Feuille de maladie

1. Le médecin crée la feuille et passe à `ISSUED`.
2. L'agent soumet la feuille, elle passe à `SUBMITTED` puis `UNDER_REVIEW`.
3. Après validation, la feuille passe à `APPROVED` ou `REJECTED`.
4. Si la feuille est approuvée, une demande de remboursement peut être créée.

9.5 Remboursement

1. L'agent crée la demande de remboursement.
2. Le backend calcule le montant remboursé selon la règle métier.
3. L'agent approuve ou rejette la demande.
4. En cas d'approbation, le paiement est exécuté et l'état passe à `EXECUTED`.

10. Règles métier clés

- l'assuré est un dossier métier, pas un utilisateur connecté,
- un médecin ne peut agir que sur ses propres dossiers,
- le statut d'un assuré doit être actif pour les nouvelles consultations,

- la consultation et la prescription doivent appartenir au même médecin authentifié,
- la feuille payée est verrouillée,
- le paiement est idempotent et le remboursement ne peut être exécuté qu'une seule fois.

11. Environnement de développement

11.1 Backend

Le backend utilise Maven et Spring Boot. Chaque service contient un `Dockerfile` et un module Maven.

Le dépôt principal contient un `pom.xml` parent qui gère les versions.

11.2 Frontend

Le frontend se trouve dans le dossier `frontend` et s'installe avec :

```
cd frontend
npm install
```

Lancer le frontend localement :

```
npm run dev
```

11.3 Exécution avec Docker Compose

Le fichier `docker-compose.yml` démarre : - le registre de découverte Eureka, - les bases Postgres, - `auth-service`, - `profile-service`, - `medical-service`, - `reimbursement-service`, - `api-gateway`.

Exécution :

```
docker compose up --build
```

Ports exposés : - 8080 : API Gateway, - 8081 : `auth-service`, - 8082 : `profile-service`, - 8083 : `medical-service`, - 8084 : `reimbursement-service`, - 8761 : `discovery-service`.

11.4 Configuration des bases de données

Chaque service a sa base Postgres dédiée avec des comptes et mots de passe définis dans le `docker-compose.yml`.

11.5 Variables d'environnement importantes

- `VITE_API_BASE_URL` (frontend)
- `INTERNAL_SERVICE_SECRET` (`auth-service`, `profile-service`)
- `JWT_SECRET`
- `SENSITIVE_DATA_KEY`

12. Points forts du projet

- architecture microservices claire,
- séparation des domaines métier,
- API Gateway et service de découverte,
- front-end moderne et typé,
- workflow de santé complet (consultation, feuille, remboursement),
- sécurité JWT et contrôle d'accès par rôle,
- audit des actions sensibles.

13. Ce qu'il faut expliquer à l'enseignant

1. le découpage en services et la raison métier de chaque service,
2. le flux d'authentification JWT,
3. le modèle de rôle et les restrictions d'accès,
4. les transitions d'état des feuilles de maladie et des remboursements,
5. la relation entre la création de compte et la synchronisation du profil métier,
6. l'organisation du frontend et la manière dont il consomme l'API Gateway.

14. Conseils pour un cours

- commencer par l'architecture technique,
- ensuite expliquer le parcours utilisateur type (agent, médecin, assuré),
- détailler les cas d'utilisation principaux,
- enfin montrer le lien entre le frontend et les APIs.

Ce document est conçu pour donner au formateur une vision complète du projet, de son architecture, de ses règles métier et de son mode de fonctionnement.